CrossCanals Checkout Concurrency Deep Dive

1. Background

The checkout endpoint creates one or more order documents from the user's cart. Initially the API used the following architecture:

Route → Authentication → Validation Middleware (isAuthenticatedCartItems) → Controller → Start MongoDB Transaction → Create Orders → Mark Cart Inactive → Commit.

2. Original Middleware Behaviour

The middleware queried the Cart collection before the transaction:

1. Read all cartIds.
2. Ensure they belong to the authenticated user.
3. Ensure every cart document had isActiveInCart=true.
4. Attach the result to req.cartDocs.
5. Call next().

The controller trusted req.cartDocs and immediately started the transaction.

3. Why This Was Incorrect

The correctness decision ("Is this cart still valid?") was made before the transaction began. This created a classic Time Of Check To Time Of Use (TOCTOU) race condition.

TOCTOU Timeline

Request A
• Middleware reads cart (active=true)
• Waits before transaction

Request B
• Middleware also reads cart (active=true)
• Waits before transaction

Now both requests have independently concluded that the cart is valid. Neither request knows about the other because neither has modified the database yet.

Both controllers then execute:
Start Transaction → Build Orders → insertMany() → updateMany(cart inactive) → Commit

Depending on timing, both transactions can commit, producing duplicate orders. The validation result became stale between the moment it was checked and the moment it was used.

4. Why MongoDB Transactions Did Not Fully Protect The Original Design

MongoDB transactions only guarantee isolation for operations executed inside the transaction. Anything executed before session.startTransaction() is outside transactional guarantees. Therefore the middleware read was never protected.

A common misconception is that simply wrapping writes in a transaction automatically removes race conditions. It does not. Transactions cannot protect decisions that were made before the transaction started.

5. Symptoms Observed During Testing

Browser testing occasionally produced duplicate orders. Stress testing showed multiple successful HTTP 201 responses for identical checkout payloads. Both requests contained identical cartIds and represented the same checkout attempt.

6. Refactored Architecture

The middleware responsible for reading cart documents was removed. The controller now starts the MongoDB transaction first and performs the cart lookup using the transaction session.

New flow:
Start Session
Start Transaction
Read Active Cart (.session(session))
Validate Cart
Business Logic
Insert Orders
Mark Cart Inactive
Commit

7. Why The New Design Works

Now both the validation and all writes execute inside the same transaction. The decision and the state transition belong to one atomic unit. If another transaction modifies the same cart while this transaction is executing, MongoDB detects the conflict and aborts one transaction.

8. Validation vs WriteConflict

The validation and WriteConflict solve different problems.

Validation answers:
• Does the cart exist?
• Does it belong to this user?
• Is it currently active?

WriteConflict answers:
• Did another concurrent transaction modify the same data after validation but before commit?

Validation checks whether the request is logically valid. WriteConflict protects against concurrent execution. Both are required.

9. Error Handling

Business errors (ServerError) are preserved and returned unchanged. Unexpected runtime/database failures become HTTP 500. MongoDB WriteConflict is translated into HTTP 409 Conflict because the request failed due to concurrent modification rather than an internal server failure.

10. Stress Test Results

A Node.js script fired 20 concurrent POST /order requests with the same payload. After moving validation inside the transaction the observed behaviour became:

• 1 request returned HTTP 201 Created.
• Remaining requests returned HTTP 409 Conflict.
• Only one order document was committed.

11. Remaining Production Improvements

Although significantly improved, an even stronger design would atomically 'claim' the cart (update isActiveInCart=false and verify modifiedCount) before performing expensive business logic. Additionally, implementing Idempotency Keys would protect against duplicate retries caused by network failures or browser resubmissions.

12. Interview Summary

Problem: Checkout validation occurred outside the transaction causing a TOCTOU race condition.

Root Cause: Multiple concurrent requests validated the same active cart before any transaction committed.

Solution: Move validation inside the MongoDB transaction so validation and writes execute atomically.

Result: Duplicate order creation under concurrent requests was eliminated during stress testing.